IPython

The Best-in-Class Python Interactive Interpreter

Python's object-oriented approach, apart from its robustness, clear syntax, developer friendliness and dynamic nature, has made it a very successful programming language. Though the standard Python interactive interpreter is unquestionably useful, Fernando Perez has gone a step further by creating IPython, an enhanced interactive Python shell, which has added history caching, object information, magic functions, tab completion and debugging to the basic interactive interpreter. IPython can also act as a system-cum-Python interactive shell. Interested? Let's roll!

Released under a BSD licence, IPython is packaged by many Linux distributions. Install it on Ubuntu using *sudo apt-get install ipython*. You can also download the latest version from *http://ipython.org/download.html*. Untar or unzip the archive and install it with *python setup.py install*. To check the version, run *ipython -Version*.

The IPython interactive shell

To start IPython, just run *ipython* in the terminal. An example of a session is shown in Figure 1. You may wonder what *'In'* and *'Out'* are. The beauty of this shell is that it keeps a list of your input and output operations; *'In'* is a list, and *'Out'* is a *dict*, as you'll see if you run *type(In)* and *type(Out)*. In IPython, the *'print'* statement is considered as a raw type, and its output is not saved to the *'Out'* dictionary—as you can see in Figure 1.

Note: While the latest version of IPython is 0.12, I'm using 0.10 on Ubuntu 10.04 32-bit. The information in this article is based on this configuration.

Tab completion and shell commands

Unlike the normal Python interpreter, you get syntax tab completion, as well as common shell key-bindings like *<Ctrl-A>* (start of line), *<Ctrl-K>* (delete line), etc. You can also mix Python code with system shell commands—just prefix shell commands with a '!' (the exclamation mark). You can store the output of shell commands in Python variables, and then manipulate them with built-in functions like *'grep'*, *'fields'*, etc, as in Figure 2. Here I assigned the output of *'ps aux'* to a *'test'* variable, and then used *grep* to get a particular process' data, and *'fields'* to display the second column of the output (index starts from zero).

What if you want to use Python variables in shell commands? Easy—use the '$' sign, like as follows:

```
user='ankur'
!ps aux|grep $user
```

Magic functions

IPython's incredible power includes a set of built-in 'magic' functions, whose names begin with '%'. Example: *%alias*, *%cd*, etc. Parameters to these are given without parentheses or quotes. To list the available magic functions, use *lsmagic*, like in Figure 3. The *Automagic* setting, if *ON* (it's ON by default), removes the need to prefix the '%'. You can make changes in the config file in the *~/.ipython* folder. I can't explain every magic function, but will cover a few in the table below.

You can get details about a magic function by using '?' with it; for example, *%alias ?* will show its package, base